```r
knitr::opts_knit$set(root.dir = here::here())
pacman::p_load(
  dplyr, tidyr, readr, lme4, lmerTest,
  emmeans, here, clubSandwich, purrr, data.table
)
source("R/metrics.R")
source("R/models.R")
source("R/posterior_helper.R")

PARAM_COLS <- c(paste0("p8_", 1:3), paste0("q_", 1:6), "N9", "v9")
MAIN_PARAMS <- c(p8_1 = "Integrated", p8_2 = "Heuristic", p8_3 = "Random", N9 = "N", v9 = "v")
Q_LABELS <- c(q_1 = "BO", q_2 = "HO", q_3 = "FO", q_4 = "JO", q_5 = "LS", q_6 = "50%")

raw <- read_csv(here::here("data", "Exp_2.csv"), show_col_types = FALSE)
order_metadata <- raw |>
  filter(block_num == 1) |>
  transmute(subject_id = as.character(subject_id), block_order = if_else(block_type == "Self-paced", "SP
  distinct()

mf <- raw |>
  transmute(
    subject_id = as.character(subject_id),
    format = factor(format, levels = c("frequency", "probability")),
    time_pressure = factor(block_type, levels = c("Self-paced", "Time-limited")),
    block_num,
    judgment = inputvalue / 100,
    bayes = correctAnswer / 100,
    BO = br, HO = hr, FO = 1 - far, JO = br * hr, LS = hr - far
  ) |>
  left_join(order_metadata, by = "subject_id") |>
  mutate(
    block_order = factor(block_order, levels = c("SP-first", "TL-first")),
    TP = as.integer(time_pressure == "Time-limited"),
    abs_error = abs(judgment - bayes),
    match_Bayes = round(judgment, 2) == round(bayes, 2),
    match_BO = round(judgment, 2) == round(BO, 2),
    match_HO = round(judgment, 2) == round(HO, 2),
    match_FO = round(judgment, 2) == round(FO, 2),
    match_JO = round(judgment, 2) == round(JO, 2),
    match_LS = round(judgment, 2) == round(LS, 2),
    match_50 = round(judgment, 2) == 0.5,
    exact_any = match_Bayes | match_BO | match_HO | match_FO | match_JO | match_LS | match_50
  )
cat("Two-block behavioural sample:", n_distinct(mf$subject_id), "participants;", nrow(mf), "trials\n")
```

```
## Two-block behavioural sample: 83 participants; 8596 trials
```

```r
fit_two_block <- function(formula, fallback_corr, fallback_simple, data, label) {
  model <- tryCatch(lmer(formula, data = data), error = function(e) NULL)
  if (is.null(model) || isSingular(model)) {
    message(label, ": failed or singular; trying no-correlation structure.")
    model <- tryCatch(lmer(fallback_corr, data = data), error = function(e) NULL)
  }
```

```
  if (is.null(model) || isSingular(model)) {
    message(label, ": still failed or singular; using random-intercept-only.")
    model <- tryCatch(lmer(fallback_simple, data = data), error = function(e) NULL)
  }
  if (is.null(model) || isSingular(model)) {
    message(label, ": reduced model remains singular; inspect before reporting.")
  }
  model
}

m_judgment <- fit_two_block(
  judgment ~ format * time_pressure + block_order * time_pressure + (1 + TP | subject_id),
  judgment ~ format * time_pressure + block_order * time_pressure + (1 + TP || subject_id),
  judgment ~ format * time_pressure + block_order * time_pressure + (1 | subject_id),
  mf, "Judgment"
)
judgment_anova <- anova(m_judgment, type = 3)
judgment_emm <- emmeans(m_judgment, ~ time_pressure | format)
```

```
## Note: D.f. calculations have been disabled because the number of observations exceeds 5000.
## To enable adjustments, add the argument 'pbkrtest.limit = 8596' (or larger)
## [or, globally, 'set emm_options(pbkrtest.limit = 8596)' or larger];
## but be warned that this may result in large computation time and memory use.
```

```
## Note: D.f. calculations have been disabled because the number of observations exceeds 5000.
## To enable adjustments, add the argument 'lmerTest.limit = 8596' (or larger)
## [or, globally, 'set emm_options(lmerTest.limit = 8596)' or larger];
## but be warned that this may result in large computation time and memory use.
```

```
judgment_contrasts <- contrast(judgment_emm, method = list("TL - SP" = c(-1, 1)))

cat("\nJudgment model:\n")
```

```
##
## Judgment model:
```

```
print(judgment_anova)
```

```
## Type III Analysis of Variance Table with Satterthwaite's method
##                             Sum Sq  Mean Sq NumDF   DenDF F value    Pr(>F)
## format                    0.092624 0.092624     1 80.1325 1.00202 0.3198365
## time_pressure             0.698930 0.698930     1 80.7363 7.56109 0.0073581 **
## block_order               0.206338 0.206338     1 80.1453 2.23219 0.1390894
## format:time_pressure      0.008376 0.008376     1 80.7567 0.09061 0.7641799
## time_pressure:block_order 0.533406 0.533406     1 80.7912 5.77043 0.0185920 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
print(judgment_contrasts)
```

```
## format = frequency:
##  contrast estimate     SE  df z.ratio p.value
##  TL - SP   -0.0242 0.0152 Inf  -1.594  0.1110
##
## format = probability:
##  contrast estimate     SE  df z.ratio p.value
##  TL - SP   -0.0301 0.0126 Inf  -2.387  0.0170
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: asymptotic
```

```
m_error <- fit_two_block(
  abs_error ~ format * time_pressure + block_order * time_pressure + (1 + TP | subject_id),
  abs_error ~ format * time_pressure + block_order * time_pressure + (1 + TP || subject_id),
  abs_error ~ format * time_pressure + block_order * time_pressure + (1 | subject_id),
  mf, "Absolute error"
)
error_anova <- anova(m_error, type = 3)
error_emm <- emmeans(m_error, ~ time_pressure | format)
```

```
## Note: D.f. calculations have been disabled because the number of observations exceeds 5000.
## To enable adjustments, add the argument 'pbkrtest.limit = 8596' (or larger)
## [or, globally, 'set emm_options(pbkrtest.limit = 8596)' or larger];
## but be warned that this may result in large computation time and memory use.
## Note: D.f. calculations have been disabled because the number of observations exceeds 5000.
## To enable adjustments, add the argument 'lmerTest.limit = 8596' (or larger)
## [or, globally, 'set emm_options(lmerTest.limit = 8596)' or larger];
## but be warned that this may result in large computation time and memory use.
```

```
error_contrasts <- contrast(error_emm, method = list("TL - SP" = c(-1, 1)))
cat("\nAbsolute error model:\n")
```

```
##
## Absolute error model:
```

```
print(error_anova)
```

```
## Type III Analysis of Variance Table with Satterthwaite's method
##                             Sum Sq  Mean Sq NumDF   DenDF  F value    Pr(>F)
## format                    0.119994 0.119994     1 79.8757  2.67445 0.1059056
## time_pressure             0.574503 0.574503     1 80.2163 12.80463 0.0005901 ***
## block_order               0.360132 0.360132     1 79.8882  8.02669 0.0058328 **
## format:time_pressure      0.278910 0.278910     1 80.2109  6.21641 0.0147147 *
## time_pressure:block_order 0.043775 0.043775     1 80.2484  0.97567 0.3262387
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
print(error_contrasts)
```

```
## format = frequency:
##  contrast estimate     SE  df z.ratio p.value
```

```
##  TL - SP    0.0793 0.0201 Inf   3.948  0.0001
##
## format = probability:
##  contrast estimate     SE  df z.ratio p.value
##  TL - SP    0.0142 0.0167 Inf   0.852  0.3944
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: asymptotic
```

```
individual_slopes <- mf |>
  group_by(subject_id, format, time_pressure) |>
  group_modify(~ {
    fit <- lm(judgment ~ bayes, data = .x)
    tibble(intercept = unname(coef(fit)[1]), slope = unname(coef(fit)[2]))
  }) |>
  ungroup()
individual_slope_summary <- individual_slopes |>
  group_by(format, time_pressure) |>
  summarise(n = n(), mean = mean(slope), sd = sd(slope), median = median(slope), .groups = "drop")
cat("\nIndividual posterior slopes:\n")
```

```
##
## Individual posterior slopes:
```

```
print(individual_slope_summary)
```

```
## # A tibble: 4 x 6
##   format      time_pressure     n  mean    sd median
##   <fct>       <fct>         <int> <dbl> <dbl>  <dbl>
## 1 frequency   Self-paced       34 0.662 0.289  0.703
## 2 frequency   Time-limited     34 0.457 0.362  0.439
## 3 probability Self-paced       49 0.452 0.401  0.425
## 4 probability Time-limited     49 0.433 0.350  0.329
```

```
m_conservatism <- lmer(
  judgment ~ bayes * format * time_pressure +
    block_order * time_pressure +
    (1 + bayes * TP | subject_id),
  data = mf,
  control = lmerControl(
    optimizer = "bobyqa",
    optCtrl = list(maxfun = 2e5)
  )
)

conservatism_anova <- anova(m_conservatism, type = 3)
conservatism_slopes <- emtrends(m_conservatism, ~ format * time_pressure, var = "bayes")
```

```
## Note: D.f. calculations have been disabled because the number of observations exceeds 5000.
## To enable adjustments, add the argument 'pbkrtest.limit = 8596' (or larger)
## [or, globally, 'set emm_options(pbkrtest.limit = 8596)' or larger];
## but be warned that this may result in large computation time and memory use.
```

```
## Note: D.f. calculations have been disabled because the number of observations exceeds 5000.
## To enable adjustments, add the argument 'lmerTest.limit = 8596' (or larger)
## [or, globally, 'set emm_options(lmerTest.limit = 8596)' or larger];
## but be warned that this may result in large computation time and memory use.
```

```
conservatism_contrasts <- contrast(conservatism_slopes, method = list("TL - SP" = c(-1, 1)), by = "forma
cat("\nPosterior sensitivity:\n")
```

```
##
## Posterior sensitivity:
```

```
print(conservatism_anova)
```

```
## Type III Analysis of Variance Table with Satterthwaite's method
##                              Sum Sq Mean Sq NumDF   DenDF   F value   Pr(>F)
## bayes                       6.22095 6.22095     1 80.9816 196.78794  < 2e-16 ***
## format                      0.09032 0.09032     1 79.5348   2.85701 0.094893 .
## time_pressure               0.09315 0.09315     1 80.7659   2.94663 0.089890 .
## block_order                 0.15648 0.15648     1 80.0013   4.94989 0.028909 *
## bayes:format                0.08438 0.08438     1 80.9816   2.66933 0.106181
## bayes:time_pressure         0.31312 0.31312     1 81.1257   9.90489 0.002306 **
## format:time_pressure        0.20780 0.20780     1 80.6985   6.57339 0.012207 *
## time_pressure:block_order   0.16832 0.16832     1 80.4400   5.32454 0.023598 *
## bayes:format:time_pressure 0.21633 0.21633     1 81.1257   6.84311 0.010609 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
print(conservatism_slopes)
```

```
##  format      time_pressure bayes.trend     SE  df asymp.LCL asymp.UCL
##  frequency   Self-paced          0.662 0.0616 Inf     0.541     0.782
##  probability Self-paced          0.452 0.0513 Inf     0.351     0.553
##  frequency   Time-limited        0.457 0.0610 Inf     0.337     0.576
##  probability Time-limited        0.433 0.0508 Inf     0.333     0.533
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: asymptotic
## Confidence level used: 0.95
```

```
print(conservatism_contrasts)
```

```
## format = frequency:
##  contrast estimate     SE  df z.ratio p.value
##  TL - SP   -0.2050 0.0546 Inf  -3.751  0.0002
##
## format = probability:
##  contrast estimate     SE  df z.ratio p.value
##  TL - SP   -0.0189 0.0455 Inf  -0.415  0.6782
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: asymptotic
```

```
conservatism_slopes_sat <- emmeans::emtrends(
  m_conservatism,
  ~ format * time_pressure,
  var = "bayes",
  lmer.df = "satterthwaite",
  lmerTest.limit = 10000
)

cat("\nPosterior slopes tested against 1:\n")
```

```
##
## Posterior slopes tested against 1:
```

```
print(summary(
  conservatism_slopes_sat,
  null = 1,
  infer = c(TRUE, TRUE)
))
```

```
##  format      time_pressure bayes.trend     SE   df lower.CL upper.CL null t.ratio p.value
##  frequency   Self-paced          0.662 0.0616 81.0    0.539    0.784    1  -5.492  <.0001
##  probability Self-paced          0.452 0.0513 81.0    0.350    0.554    1 -10.680  <.0001
##  frequency   Time-limited        0.457 0.0610 81.0    0.335    0.578    1  -8.914  <.0001
##  probability Time-limited        0.433 0.0508 81.1    0.332    0.534    1 -11.162  <.0001
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: satterthwaite
## Confidence level used: 0.95
```

```
mf_nonexact <- mf |> filter(!exact_any)
m_nonexact <- lmer(
  judgment ~ bayes * format * time_pressure +
    block_order * time_pressure +
    (1 + bayes * TP | subject_id),
  data = mf_nonexact,
  control = lmerControl(
    optimizer = "bobyqa",
    optCtrl = list(maxfun = 2e5)
  )
)

nonexact_anova <- anova(m_nonexact, type = 3)
nonexact_slopes <- emtrends(m_nonexact, ~ format * time_pressure, var = "bayes")
nonexact_contrasts <- contrast(nonexact_slopes, method = list("TL - SP" = c(-1, 1)), by = "format")
cat("\nNon-exact responses:\n")
```

```
##
## Non-exact responses:
```

```
print(nonexact_anova)
```

```
## Type III Analysis of Variance Table with Satterthwaite's method
##                              Sum Sq Mean Sq NumDF   DenDF   F value     Pr(>F)
## bayes                       3.96681 3.96681     1 67.4678 103.90968 2.7105e-15 ***
## format                      0.00205 0.00205     1 65.7458   0.05378  0.8173303
## time_pressure               0.16089 0.16089     1 53.8825   4.21436  0.0449524 *
## block_order                 0.07514 0.07514     1 72.2624   1.96817  0.1649247
## bayes:format                0.00042 0.00042     1 67.4602   0.01097  0.9169134
## bayes:time_pressure         0.41284 0.41284     1 59.7849  10.81422  0.0016905 **
## format:time_pressure        0.06386 0.06386     1 53.7288   1.67270  0.2014303
## time_pressure:block_order   0.12984 0.12984     1 60.4855   3.40102  0.0700550 .
## bayes:format:time_pressure 0.13879 0.13879     1 59.7507   3.63546  0.0613730 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
print(nonexact_slopes)
```

```
##  format      time_pressure bayes.trend     SE   df lower.CL upper.CL
##  frequency   Self-paced          0.485 0.0642 67.4    0.357    0.613
##  probability Self-paced          0.411 0.0585 70.6    0.294    0.528
##  frequency   Time-limited        0.304 0.0606 68.4    0.183    0.425
##  probability Time-limited        0.363 0.0552 72.2    0.253    0.473
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: kenward-roger
## Confidence level used: 0.95
```

```
print(nonexact_contrasts)
```

```
## format = frequency:
##  contrast estimate     SE   df t.ratio p.value
##  TL - SP   -0.1810 0.0516 62.3  -3.511  0.0008
##
## format = probability:
##  contrast estimate     SE   df t.ratio p.value
##  TL - SP   -0.0482 0.0475 62.1  -1.014  0.3145
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: kenward-roger
```

```
match_cols <- c("match_Bayes", "match_BO", "match_HO", "match_FO",
                "match_JO", "match_LS", "match_50")

match_trial <- mf |>
  mutate(
    n_matches = rowSums(across(all_of(match_cols))),
    across(all_of(match_cols), ~ if_else(n_matches > 0, as.numeric(.x) / n_matches, 0))
  )

exact_match_table <- match_trial |>
  group_by(format, time_pressure) |>
  summarise(across(all_of(match_cols), mean), .groups = "drop") |>
  mutate(Heuristic = match_BO + match_HO + match_FO + match_JO + match_LS + match_50) |>
```

```
  transmute(
    format, time_pressure,
    BO = match_BO, REP = match_HO, FC = match_FO,
    JO = match_JO, LS = match_LS, `50%` = match_50,
    Heuristic, Bayes = match_Bayes
  )
cat("\nFractional-credit exact-match table:\n")
```

```
##
## Fractional-credit exact-match table:
```

```
print(exact_match_table |> mutate(across(where(is.numeric), ~ round(.x, 2))), n = Inf)
```

```
## # A tibble: 4 x 10
##   format      time_pressure    BO   REP    FC    JO    LS `50%` Heuristic Bayes
##   <fct>       <fct>         <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>     <dbl> <dbl>
## 1 frequency   Self-paced     0.15  0.04  0     0.07  0     0.01      0.27  0.19
## 2 frequency   Time-limited   0.13  0.03  0.01  0.08  0     0.02      0.26  0.08
## 3 probability Self-paced     0.11  0.15  0.06  0.05  0.03  0.02      0.42  0.11
## 4 probability Time-limited   0.14  0.12  0.03  0.06  0.01  0.06      0.42  0.05
```

```
match_props <- match_trial |>
  dplyr::select(subject_id, format, time_pressure, block_order, TP, all_of(match_cols)) |>
  pivot_longer(all_of(match_cols), names_to = "match_type", values_to = "match") |>
  mutate(match_type = dplyr::recode(match_type,
                                    match_Bayes = "Bayes", match_BO = "BO", match_HO = "HO",
                                    match_FO = "FO", match_JO = "JO", match_LS = "LS",
                                    match_50 = "50%")) |>
  group_by(subject_id, format, time_pressure, block_order, TP, match_type) |>
  summarise(match_prop = mean(match), .groups = "drop")

match_types <- c("Bayes", "BO", "HO", "FO", "JO", "LS", "50%")

match_models <- lapply(match_types, function(mt) {
  data <- match_props |> filter(match_type == mt)
  fit_two_block(
    match_prop ~ format * time_pressure + block_order * time_pressure + (1 + TP | subject_id),
    match_prop ~ format * time_pressure + block_order * time_pressure + (1 + TP || subject_id),
    match_prop ~ format * time_pressure + block_order * time_pressure + (1 | subject_id),
    data, paste("Exact matches:", mt)
  )
})
```

```
## Exact matches: Bayes: failed or singular; trying no-correlation structure.
```

```
## boundary (singular) fit: see help('isSingular')
```

```
## Exact matches: Bayes: still failed or singular; using random-intercept-only.
```

```
## Exact matches: BO: failed or singular; trying no-correlation structure.
```

```
## boundary (singular) fit: see help('isSingular')

## Exact matches: BO: still failed or singular; using random-intercept-only.

## Exact matches: HO: failed or singular; trying no-correlation structure.

## boundary (singular) fit: see help('isSingular')

## Exact matches: HO: still failed or singular; using random-intercept-only.

## Exact matches: FO: failed or singular; trying no-correlation structure.

## boundary (singular) fit: see help('isSingular')

## Exact matches: FO: still failed or singular; using random-intercept-only.

## Exact matches: JO: failed or singular; trying no-correlation structure.

## Exact matches: LS: failed or singular; trying no-correlation structure.

## boundary (singular) fit: see help('isSingular')

## Exact matches: LS: still failed or singular; using random-intercept-only.

## Exact matches: 50%: failed or singular; trying no-correlation structure.
```

```r
names(match_models) <- match_types

for (mt in match_types) {
  cat("\n===== Exact matches:", mt, "=====\n")
  model <- match_models[[mt]]
  print(anova(model, type = 3))
  emm <- emmeans(model, ~ time_pressure | format)
  print(contrast(emm, method = list("TL - SP" = c(-1, 1))))
  if (isSingular(model)) {
    m_simple <- lm(match_prop ~ format * time_pressure + block_order * time_pressure,
                   data = match_props |> filter(match_type == mt))
    robust <- clubSandwich::coef_test(
      m_simple, vcov = "CR2",
      cluster = (match_props |> filter(match_type == mt))$subject_id,
      test = "Satterthwaite"
    )
    print(robust)
  }
}
```

```
##
## ===== Exact matches: Bayes =====
## Type III Analysis of Variance Table with Satterthwaite's method
##                              Sum Sq   Mean Sq NumDF DenDF  F value     Pr(>F)
```

```
## format                    0.0566974 0.0566974     1    80  3.36378 0.07036343 .
## time_pressure             0.2837407 0.2837407     1    80 16.83392 9.7527e-05 ***
## block_order               0.2771788 0.2771788     1    80 16.44461 0.00011557 ***
## format:time_pressure      0.0276448 0.0276448     1    80  1.64013 0.20400837
## time_pressure:block_order 0.0696707 0.0696707     1    80  4.13346 0.04535972 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## format = frequency:
##  contrast estimate     SE df t.ratio p.value
##  TL - SP    -0.111 0.0316 80  -3.501  0.0008
##
## format = probability:
##  contrast estimate     SE df t.ratio p.value
##  TL - SP    -0.058 0.0262 80  -2.211  0.0299
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: kenward-roger
##
## ===== Exact matches: BO =====
## Type III Analysis of Variance Table with Satterthwaite's method
##                             Sum Sq   Mean Sq NumDF DenDF F value   Pr(>F)
## format                    0.0014943 0.0014943     1    80 0.05455 0.815924
## time_pressure             0.0000440 0.0000440     1    80 0.00160 0.968149
## block_order               0.1131243 0.1131243     1    80 4.12970 0.045456 *
## format:time_pressure      0.0155259 0.0155259     1    80 0.56678 0.453749
## time_pressure:block_order 0.0072571 0.0072571     1    80 0.26493 0.608174
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## format = frequency:
##  contrast estimate     SE df t.ratio p.value
##  TL - SP   -0.0186 0.0403 80  -0.463  0.6446
##
## format = probability:
##  contrast estimate     SE df t.ratio p.value
##  TL - SP    0.0207 0.0334 80   0.620  0.5370
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: kenward-roger
##
## ===== Exact matches: HO =====
## Type III Analysis of Variance Table with Satterthwaite's method
##                               Sum Sq    Mean Sq NumDF DenDF F value   Pr(>F)
## format                    0.02424251 0.02424251     1    80 3.42581 0.067876 .
## time_pressure             0.01765997 0.01765997     1    80 2.49560 0.118111
## block_order               0.00122539 0.00122539     1    80 0.17317 0.678428
## format:time_pressure      0.00319429 0.00319429     1    80 0.45140 0.503607
## time_pressure:block_order 0.00014436 0.00014436     1    80 0.02040 0.886785
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## format = frequency:
##  contrast estimate     SE df t.ratio p.value
##  TL - SP   -0.0121 0.0205 80  -0.591  0.5562
##
## format = probability:
```

```
##  contrast estimate     SE df t.ratio p.value
##  TL - SP   -0.0300 0.0170 80  -1.762  0.0819
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: kenward-roger
##
## ===== Exact matches: FO =====
## Type III Analysis of Variance Table with Satterthwaite's method
##                               Sum Sq    Mean Sq NumDF DenDF F value  Pr(>F)
## format                     0.01758792 0.01758792     1    80 1.66878 0.20014
## time_pressure              0.01001177 0.01001177     1    80 0.94994 0.33267
## block_order                0.00000203 0.00000203     1    80 0.00019 0.98897
## format:time_pressure       0.01963089 0.01963089     1    80 1.86263 0.17615
## time_pressure:block_order 0.02220321 0.02220321     1    80 2.10669 0.15057
## format = frequency:
##  contrast estimate     SE df t.ratio p.value
##  TL - SP   0.00631 0.0250 80   0.253  0.8011
##
## format = probability:
##  contrast estimate     SE df t.ratio p.value
##  TL - SP  -0.03797 0.0207 80  -1.830  0.0709
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: kenward-roger
##
## ===== Exact matches: JO =====
## Type III Analysis of Variance Table with Satterthwaite's method
##                               Sum Sq     Mean Sq NumDF   DenDF F value  Pr(>F)
## format                     0.000833171 0.000833171     1 80.0041 0.65321 0.42137
## time_pressure              0.001818731 0.001818731     1 79.9999 1.42589 0.23597
## block_order                0.000146614 0.000146614     1 80.0041 0.11495 0.73547
## format:time_pressure       0.000180054 0.000180054     1 79.9999 0.14116 0.70812
## time_pressure:block_order 0.000021245 0.000021245     1 79.9999 0.01666 0.89764
## format = frequency:
##  contrast estimate     SE df t.ratio p.value
##  TL - SP   0.01435 0.0141 80   1.021  0.3105
##
## format = probability:
##  contrast estimate     SE df t.ratio p.value
##  TL - SP   0.00749 0.0117 80   0.641  0.5233
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: kenward-roger
##
## ===== Exact matches: LS =====
## Type III Analysis of Variance Table with Satterthwaite's method
##                               Sum Sq    Mean Sq NumDF DenDF F value  Pr(>F)
## format                     0.01107663 0.01107663     1    80 1.77873 0.18609
## time_pressure              0.00060126 0.00060126     1    80 0.09655 0.75681
## block_order                0.00001470 0.00001470     1    80 0.00236 0.96137
## format:time_pressure       0.00227500 0.00227500     1    80 0.36533 0.54727
## time_pressure:block_order 0.01053068 0.01053068     1    80 1.69106 0.19719
## format = frequency:
##  contrast estimate     SE df t.ratio p.value
```

```
##  TL - SP   0.00366 0.0192 80   0.191  0.8494
##
## format = probability:
##  contrast estimate     SE df t.ratio p.value
##  TL - SP  -0.01142 0.0159 80  -0.716  0.4761
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: kenward-roger
##
## ===== Exact matches: 50% =====
## Type III Analysis of Variance Table with Satterthwaite's method
##                                Sum Sq     Mean Sq NumDF   DenDF F value   Pr(>F)
## format                     0.001522404 0.001522404     1 80.0030 3.81659 0.054243 .
## time_pressure              0.001133468 0.001133468     1 80.0004 2.84155 0.095753 .
## block_order                0.001562841 0.001562841     1 80.0030 3.91796 0.051212 .
## format:time_pressure       0.000663244 0.000663244     1 80.0004 1.66272 0.200952
## time_pressure:block_order 0.002029247 0.002029247     1 80.0004 5.08722 0.026834 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## format = frequency:
##  contrast estimate     SE df t.ratio p.value
##  TL - SP   0.00509 0.0197 80   0.259  0.7965
##
## format = probability:
##  contrast estimate     SE df t.ratio p.value
##  TL - SP   0.03808 0.0164 80   2.328  0.0224
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: kenward-roger
```

```
heuristic_props <- match_trial |>
  mutate(
    heuristic_match =
      match_BO + match_HO + match_FO +
      match_JO + match_LS + match_50
  ) |>
  group_by(
    subject_id, format, time_pressure,
    block_order, TP
  ) |>
  summarise(
    match_prop = mean(heuristic_match),
    .groups = "drop"
  )

m_heuristic_order <- fit_two_block(
  match_prop ~ format * time_pressure +
    block_order * time_pressure +
    (1 + TP | subject_id),

  match_prop ~ format * time_pressure +
    block_order * time_pressure +
    (1 + TP || subject_id),
```

```
  match_prop ~ format * time_pressure +
    block_order * time_pressure +
    (1 | subject_id),

  heuristic_props,
  "Overall heuristic matching"
)
```

```
## Overall heuristic matching: failed or singular; trying no-correlation structure.
```

```
## boundary (singular) fit: see help('isSingular')
```

```
## Overall heuristic matching: still failed or singular; using random-intercept-only.
```

```
cat("\nOverall heuristic matching and block order:\n")
```

```
##
## Overall heuristic matching and block order:
```

```
print(anova(m_heuristic_order, type = 3))
```

```
## Type III Analysis of Variance Table with Satterthwaite's method
##                              Sum Sq   Mean Sq NumDF DenDF F value   Pr(>F)
## format                     0.1563343 0.1563343     1    80 4.62745 0.034482 *
## time_pressure              0.0020574 0.0020574     1    80 0.06090 0.805713
## block_order                0.0712694 0.0712694     1    80 2.10955 0.150293
## format:time_pressure       0.0013724 0.0013724     1    80 0.04062 0.840779
## time_pressure:block_order  0.0004291 0.0004291     1    80 0.01270 0.910548
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
posterior_path <- here::here("data", "Exp2_ML_Posterior_Draws_r05.rds")
posterior_draws <- readRDS(posterior_path)
block_keys <- raw |>
  transmute(subject_id = as.character(subject_id), subject_block = paste(subject_id, block_type, sep =
            format = as.character(format), time_pressure = as.character(block_type), block_num) |>
  distinct() |>
  left_join(order_metadata, by = "subject_id")
posterior_medians <- posterior_draws |>
  semi_join(block_keys, by = "subject_block") |>
  group_by(subject_block) |>
  summarise(across(all_of(PARAM_COLS), ~ median(.x, na.rm = TRUE)), .groups = "drop") |>
  left_join(block_keys, by = "subject_block") |>
  mutate(format = factor(format, levels = c("frequency", "probability")),
         time_pressure = factor(time_pressure, levels = c("Self-paced", "Time-limited")),
         block_order = factor(block_order, levels = c("SP-first", "TL-first")),
         TP = as.integer(time_pressure == "Time-limited"))
complete_subjects <- posterior_medians |>
  dplyr::count(subject_id) |> filter(n == 2) |> pull(subject_id)
posterior_medians <- posterior_medians |> filter(subject_id %in% complete_subjects)
cat("Two-block ABC sample:", n_distinct(posterior_medians$subject_id), "participants\n")
```

```
## Two-block ABC sample: 83 participants
```

```
parameter_long <- posterior_medians |>
  pivot_longer(all_of(PARAM_COLS), names_to = "parameter", values_to = "value")
describe_params <- function(data, group_col = "parameter") {
  data |>
    group_by(across(all_of(c(group_col, "format", "time_pressure")))) |>
    summarise(n = n(), mean = mean(value), sd = sd(value), median = median(value), .groups = "drop")
}
paired_test <- function(data, group_col = "parameter") {
  data |>
    group_by(across(all_of(c(group_col, "format")))) |>
    group_modify(~ {
      wide <- .x |> dplyr::select(subject_id, time_pressure, value) |>
        pivot_wider(names_from = time_pressure, values_from = value)
      tt <- t.test(wide$`Time-limited`, wide$`Self-paced`, paired = TRUE)
      tibble(n = nrow(wide), mean_SP = mean(wide$`Self-paced`), mean_TL = mean(wide$`Time-limited`),
             difference_TL_minus_SP = mean(wide$`Time-limited` - wide$`Self-paced`),
             t = unname(tt$statistic), df = unname(tt$parameter), p = tt$p.value,
             CI_low = tt$conf.int[1], CI_high = tt$conf.int[2])
    }) |> ungroup()
}
main_long <- parameter_long |>
  filter(parameter %in% names(MAIN_PARAMS)) |>
  mutate(parameter = dplyr::recode(parameter, !!!MAIN_PARAMS))

q_long <- parameter_long |>
  filter(parameter %in% names(Q_LABELS)) |>
  mutate(heuristic = dplyr::recode(parameter, !!!Q_LABELS))

main_parameter_descriptives <- describe_params(main_long)
main_parameter_tests <- paired_test(main_long)

q_descriptives <- describe_params(q_long, "heuristic")
q_tests <- paired_test(q_long, "heuristic")

cat("\nTwo-block main parameter descriptives:\n")
```

```
##
## Two-block main parameter descriptives:
```

```
print(main_parameter_descriptives, n = Inf)
```

```
## # A tibble: 20 x 7
##    parameter  format      time_pressure     n   mean      sd median
##    <chr>      <fct>       <fct>         <int>  <dbl>   <dbl>  <dbl>
##  1 Heuristic  frequency   Self-paced       34 0.419  0.175   0.421
##  2 Heuristic  frequency   Time-limited     34 0.465  0.164   0.489
##  3 Heuristic  probability Self-paced       49 0.527  0.206   0.524
##  4 Heuristic  probability Time-limited     49 0.526  0.184   0.541
##  5 Integrated frequency   Self-paced       34 0.556  0.176   0.557
##  6 Integrated frequency   Time-limited     34 0.508  0.164   0.484
##  7 Integrated probability Self-paced       49 0.447  0.206   0.451
```

```
##  8 Integrated probability Time-limited     49 0.448  0.183   0.432
##  9 N          frequency   Self-paced       34 3.75   0.955   4
## 10 N          frequency   Time-limited     34 3.5    1.19    3
## 11 N          probability Self-paced       49 3.78   1.10    4
## 12 N          probability Time-limited     49 3.45   0.980   3
## 13 Random     frequency   Self-paced       34 0.0260 0.00103 0.0262
## 14 Random     frequency   Time-limited     34 0.0264 0.00134 0.0264
## 15 Random     probability Self-paced       49 0.0259 0.00114 0.0261
## 16 Random     probability Time-limited     49 0.0263 0.00134 0.0263
## 17 v          frequency   Self-paced       34 0.524  0.249   0.459
## 18 v          frequency   Time-limited     34 0.514  0.224   0.476
## 19 v          probability Self-paced       49 0.505  0.179   0.501
## 20 v          probability Time-limited     49 0.548  0.189   0.509
```

```
cat("\nTwo-block paired parameter tests:\n")
```

```
##
## Two-block paired parameter tests:
```

```
print(main_parameter_tests, n = Inf)
```

```
## # A tibble: 10 x 11
##    parameter  format          n mean_SP mean_TL difference_TL_minus_SP       t    df       p     CI_l
##    <chr>      <fct>       <int>   <dbl>   <dbl>                  <dbl>   <dbl> <dbl>  <dbl>      <db
##  1 Heuristic  frequency      34  0.419   0.465                 0.0466    1.90      33 0.0661 -0.00329
##  2 Heuristic  probability    49  0.527   0.526                -0.000578 -0.0278    48 0.978  -0.0425
##  3 Integrated frequency      34  0.556   0.508                -0.0477   -1.94      33 0.0613 -0.0979
##  4 Integrated probability    49  0.447   0.448                 0.000662  0.0321    48 0.975  -0.0408
##  5 N          frequency      34  3.75    3.5                  -0.25     -1.39      33 0.173  -0.615
##  6 N          probability    49  3.78    3.45                 -0.327    -2.03      48 0.0478 -0.650
##  7 Random     frequency      34  0.0260  0.0264                0.000386  1.68      33 0.103  -0.00008
##  8 Random     probability    49  0.0259  0.0263                0.000375  1.81      48 0.0764 -0.00004
##  9 v          frequency      34  0.524   0.514                -0.00969  -0.424     33 0.674  -0.0562
## 10 v          probability    49  0.505   0.548                 0.0432    1.79      48 0.0802 -0.00540
```

```
PARAM_LABELS <- c(
  p8_1 = "P(Integrated)",
  p8_2 = "P(Heuristic)",
  p8_3 = "P(Random)",
  N9   = "N",
  v9   = "v",
  q_1  = "q_BO",
  q_2  = "q_HO",
  q_3  = "q_FO",
  q_4  = "q_JO",
  q_5  = "q_LS",
  q_6  = "q_50%"
)

full_parameter_table <- posterior_medians |>
  dplyr::select(format, time_pressure, dplyr::all_of(names(PARAM_LABELS))) |>
  tidyr::pivot_longer(
```

```
    cols = dplyr::all_of(names(PARAM_LABELS)),
    names_to = "parameter",
    values_to = "value"
  ) |>
  dplyr::group_by(parameter, format, time_pressure) |>
  dplyr::summarise(
    mean = mean(value, na.rm = TRUE),
    .groups = "drop"
  ) |>
  dplyr::mutate(
    parameter = factor(
      parameter,
      levels = names(PARAM_LABELS),
      labels = unname(PARAM_LABELS)
    ),
    condition = paste(
      as.character(format),
      ifelse(time_pressure == "Self-paced", "SP", "TL"),
      sep = "_"
    ),
    estimate = dplyr::case_when(
      parameter == "N" ~ sprintf("%.2f", mean),
      parameter == "P(Random)" ~ sprintf("%.4f", mean),
      TRUE ~ sprintf("%.3f", mean)
    )
  ) |>
  dplyr::select(parameter, condition, estimate) |>
  tidyr::pivot_wider(
    names_from = condition,
    values_from = estimate
  ) |>
  dplyr::arrange(parameter)

cat("\nAll parameter means:\n")
```

```
##
## All parameter means:
```

```
print(full_parameter_table, n = Inf)
```

```
## # A tibble: 11 x 5
##    parameter     frequency_SP frequency_TL probability_SP probability_TL
##    <fct>         <chr>        <chr>        <chr>          <chr>
##  1 P(Integrated) 0.556        0.508        0.447          0.448
##  2 P(Heuristic)  0.419        0.465        0.527          0.526
##  3 P(Random)     0.0260       0.0264       0.0259         0.0263
##  4 N             3.75         3.50         3.78           3.45
##  5 v             0.524        0.514        0.505          0.548
##  6 q_BO          0.162        0.154        0.139          0.149
##  7 q_HO          0.144        0.137        0.174          0.155
##  8 q_FO          0.111        0.110        0.133          0.108
##  9 q_JO          0.172        0.173        0.153          0.164
```

```
## 10 q_LS          0.117        0.121        0.123          0.121
## 11 q_50%         0.131        0.151        0.124          0.159
```

```r
parameter_order_models <- lapply(names(MAIN_PARAMS), function(param) {
  data <- posterior_medians |>
    mutate(value = .data[[param]])
  model <- lmer(
    value ~ format * time_pressure + block_order * time_pressure +
      (1 | subject_id),
    data = data
  )
  list(
    model = model,
    anova = anova(model, type = 3)
  )
})

names(parameter_order_models) <- names(MAIN_PARAMS)

for (param in c("p8_1", "p8_2", "p8_3", "N9")) {
  cat("\nParameter:", MAIN_PARAMS[[param]], "\n")
  model <- parameter_order_models[[param]]$model
  emm <- emmeans(
    model,
    ~ time_pressure | format,
    lmer.df = "satterthwaite"
  )
  print(summary(
    contrast(
      emm,
      method = list("TL - SP" = c(-1, 1))
    ),
    infer = TRUE
  ))
}
```

```
##
## Parameter: Integrated
## format = frequency:
##  contrast estimate     SE df lower.CL upper.CL t.ratio p.value
##  TL - SP  -0.04385 0.0243 80  -0.0921  0.00443  -1.808  0.0744
##
## format = probability:
##  contrast estimate     SE df lower.CL upper.CL t.ratio p.value
##  TL - SP   0.00134 0.0202 80  -0.0388  0.04145   0.066  0.9473
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: satterthwaite
## Confidence level used: 0.95
##
## Parameter: Heuristic
## format = frequency:
##  contrast estimate     SE df lower.CL upper.CL t.ratio p.value
##  TL - SP   0.04266 0.0243 80 -0.00577   0.0911   1.753  0.0834
```

```
##
## format = probability:
##  contrast estimate     SE df lower.CL upper.CL t.ratio p.value
##  TL - SP  -0.00127 0.0202 80 -0.04150   0.0390  -0.063  0.9502
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: satterthwaite
## Confidence level used: 0.95
##
## Parameter: Random
## format = frequency:
##  contrast estimate       SE df  lower.CL upper.CL t.ratio p.value
##  TL - SP  0.000339 0.000233 80 -1.25e-04 0.000804   1.453  0.1502
##
## format = probability:
##  contrast estimate       SE df  lower.CL upper.CL t.ratio p.value
##  TL - SP  0.000367 0.000194 80 -1.88e-05 0.000753   1.893  0.0620
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: satterthwaite
## Confidence level used: 0.95
##
## Parameter: N
## format = frequency:
##  contrast estimate    SE df lower.CL upper.CL t.ratio p.value
##  TL - SP    -0.231 0.187 80   -0.604   0.1414  -1.235  0.2204
##
## format = probability:
##  contrast estimate    SE df lower.CL upper.CL t.ratio p.value
##  TL - SP    -0.323 0.156 80   -0.633  -0.0138  -2.079  0.0409
##
## Results are averaged over the levels of: block_order
## Degrees-of-freedom method: satterthwaite
## Confidence level used: 0.95
```

```r
parameter_models_sum <- lapply(
  names(MAIN_PARAMS),
  function(param) {
    data <- posterior_medians |>
      mutate(
        value = .data[[param]],
        format = factor(format),
        time_pressure = factor(time_pressure),
        block_order = factor(block_order)
      )
    contrasts(data$format) <- contr.sum(2)
    contrasts(data$time_pressure) <- contr.sum(2)
    contrasts(data$block_order) <- contr.sum(2)
    model <- lmer(
      value ~ format * time_pressure +
        block_order * time_pressure +
        (1 | subject_id),
      data = data
    )
```

```
    anova(model, type = 3)
  }
)

names(parameter_models_sum) <- names(MAIN_PARAMS)

for (param in c("p8_1", "p8_2", "p8_3", "N9")) {
  cat("\nParameter:", MAIN_PARAMS[[param]], "\n")
  print(parameter_models_sum[[param]])
}
```

```
##
## Parameter: Integrated
## Type III Analysis of Variance Table with Satterthwaite's method
##                              Sum Sq   Mean Sq NumDF DenDF F value   Pr(>F)
## format                     0.0559446 0.0559446     1    80 5.62250 0.020136 *
## time_pressure              0.0180582 0.0180582     1    80 1.81487 0.181728
## block_order                0.0511898 0.0511898     1    80 5.14464 0.026014 *
## format:time_pressure       0.0204495 0.0204495     1    80 2.05520 0.155584
## time_pressure:block_order 0.0452338 0.0452338     1    80 4.54605 0.036062 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Parameter: Heuristic
## Type III Analysis of Variance Table with Satterthwaite's method
##                              Sum Sq   Mean Sq NumDF DenDF F value   Pr(>F)
## format                     0.0567968 0.0567968     1    80 5.67350 0.019597 *
## time_pressure              0.0171169 0.0171169     1    80 1.70983 0.194753
## block_order                0.0509697 0.0509697     1    80 5.09143 0.026773 *
## format:time_pressure       0.0193195 0.0193195     1    80 1.92985 0.168630
## time_pressure:block_order 0.0467953 0.0467953     1    80 4.67444 0.033603 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Parameter: Random
## Type III Analysis of Variance Table with Satterthwaite's method
##                                Sum Sq     Mean Sq NumDF DenDF F value   Pr(>F)
## format                     1.54860e-07 1.54860e-07     1    80 0.16812 0.682885
## time_pressure              4.98226e-06 4.98226e-06     1    80 5.40867 0.022571 *
## block_order                3.92217e-06 3.92217e-06     1    80 4.25786 0.042312 *
## format:time_pressure       7.82000e-09 7.82000e-09     1    80 0.00849 0.926802
## time_pressure:block_order 6.48897e-06 6.48897e-06     1    80 7.04434 0.009590 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Parameter: N
## Type III Analysis of Variance Table with Satterthwaite's method
##                              Sum Sq Mean Sq NumDF DenDF  F value    Pr(>F)
## format                     0.02849 0.02849     1    80  0.04809 0.8269777
## time_pressure              3.07145 3.07145     1    80  5.18403 0.0254669 *
## block_order                6.02651 6.02651     1    80 10.17162 0.0020369 **
## format:time_pressure       0.08485 0.08485     1    80  0.14322 0.7061054
## time_pressure:block_order 1.05164 1.05164     1    80  1.77498 0.1865498
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
judgment_block_emm <- emmeans(
  m_judgment,
  ~ time_pressure | block_order
)
```

```
## Note: D.f. calculations have been disabled because the number of observations exceeds 5000.
## To enable adjustments, add the argument 'pbkrtest.limit = 8596' (or larger)
## [or, globally, 'set emm_options(pbkrtest.limit = 8596)' or larger];
## but be warned that this may result in large computation time and memory use.
```

```
## Note: D.f. calculations have been disabled because the number of observations exceeds 5000.
## To enable adjustments, add the argument 'lmerTest.limit = 8596' (or larger)
## [or, globally, 'set emm_options(lmerTest.limit = 8596)' or larger];
## but be warned that this may result in large computation time and memory use.
```

```
judgment_block_contrasts <- contrast(
  judgment_block_emm,
  method = list("TL - SP" = c(-1, 1))
)

cat("\nJudgment time-pressure contrasts by block order:\n")
```

```
##
## Judgment time-pressure contrasts by block order:
```

```
print(summary(judgment_block_contrasts, infer = TRUE))
```

```
## block_order = SP-first:
##  contrast estimate     SE  df asymp.LCL asymp.UCL z.ratio p.value
##  TL - SP   -0.0038 0.0143 Inf   -0.0318    0.0243  -0.265  0.7908
##
## block_order = TL-first:
##  contrast estimate     SE  df asymp.LCL asymp.UCL z.ratio p.value
##  TL - SP   -0.0504 0.0134 Inf   -0.0766   -0.0243  -3.778  0.0002
##
## Results are averaged over the levels of: format
## Degrees-of-freedom method: asymptotic
## Confidence level used: 0.95
```

```
library(dplyr)
library(tidyr)
library(ggplot2)

valid_subjects <- unique(as.character(posterior_medians$subject_id))

order_metadata <- raw |>
  filter(
    block_num == 1,
```

```
    as.character(subject_id) %in% valid_subjects
  ) |>
  transmute(
    subject = as.character(subject_id),
    block_order = if_else(
      block_type == "Self-paced",
      "SP-first",
      "TL-first"
    )
  ) |>
  distinct()

posterior_plot_long <- posterior_medians |>
  transmute(
    subject = as.character(subject_id),
    format = as.character(format),
    time_pressure = as.character(time_pressure),
    block_num,
    block_order = as.character(block_order),
    p8_1,
    p8_2,
    p8_3,
    N9
  ) |>
  pivot_longer(
    cols = c(p8_1, p8_2, p8_3, N9),
    names_to = "measure",
    values_to = "value"
  ) |>
  mutate(
    measure = recode(
      measure,
      p8_1 = "P(Integrated)",
      p8_2 = "P(Heuristic)",
      p8_3 = "P(Random)",
      N9 = "Sample size (N)"
    ),
    block = factor(
      block_num,
      levels = c(1, 2),
      labels = c("First block", "Second block")
    )
  ) |>
  dplyr::select(
    subject, format, time_pressure,
    block_num, block_order, block,
    measure, value
  )

match_cols <- c(
  "match_Bayes",
  "match_BO",
  "match_HO",
```

```
  "match_FO",
  "match_JO",
  "match_LS",
  "match_50"
)

match_by_subject <- raw |>
  filter(as.character(subject_id) %in% valid_subjects) |>
  transmute(
    subject = as.character(subject_id),
    format = as.character(format),
    time_pressure = as.character(block_type),
    block_num,
    judgment = inputvalue / 100,
    bayes = correctAnswer / 100,
    BO = br,
    HO = hr,
    FO = 1 - far,
    JO = br * hr,
    LS = hr - far
  ) |>
  mutate(
    match_Bayes = round(judgment, 2) == round(bayes, 2),
    match_BO = round(judgment, 2) == round(BO, 2),
    match_HO = round(judgment, 2) == round(HO, 2),
    match_FO = round(judgment, 2) == round(FO, 2),
    match_JO = round(judgment, 2) == round(JO, 2),
    match_LS = round(judgment, 2) == round(LS, 2),
    match_50 = round(judgment, 2) == 0.5
  ) |>
  mutate(
    n_matches = rowSums(across(all_of(match_cols)))
  ) |>
  mutate(
    across(
      all_of(match_cols),
      ~ if_else(
        n_matches > 0,
        as.numeric(.x) / n_matches,
        0
      )
    )
  ) |>
  left_join(order_metadata, by = "subject") |>
  group_by(
    subject, format, time_pressure,
    block_num, block_order
  ) |>
  summarise(
    `Heuristic match` = mean(
      match_BO + match_HO + match_FO +
        match_JO + match_LS + match_50
    ),
```

```
    `Bayes match` = mean(match_Bayes),
    .groups = "drop"
  ) |>
  pivot_longer(
    cols = c(`Heuristic match`, `Bayes match`),
    names_to = "measure",
    values_to = "value"
  ) |>
  mutate(
    block = factor(
      block_num,
      levels = c(1, 2),
      labels = c("First block", "Second block")
    )
  ) |>
  dplyr::select(
    subject, format, time_pressure,
    block_num, block_order, block,
    measure, value
  )

measure_levels <- c(
  "P(Integrated)",
  "P(Heuristic)",
  "P(Random)",
  "Sample size (N)",
  "Heuristic match",
  "Bayes match"
)

block_plot_data <- bind_rows(
  posterior_plot_long,
  match_by_subject
) |>
  mutate(
    measure = factor(
      measure,
      levels = measure_levels
    ),
    format = factor(
      format,
      levels = c("frequency", "probability")
    ),
    time_pressure = factor(
      time_pressure,
      levels = c("Self-paced", "Time-limited")
    ),
    block_order = factor(
      block_order,
      levels = c("SP-first", "TL-first")
    ),
    block = factor(
      block,
```

```
      levels = c("First block", "Second block")
    )
  )

block_plot_summary <- block_plot_data |>
  group_by(
    format, measure, block_order,
    block, time_pressure
  ) |>
  summarise(
    n = sum(!is.na(value)),
    mean = mean(value, na.rm = TRUE),
    sd = sd(value, na.rm = TRUE),
    se = sd / sqrt(n),
    .groups = "drop"
  )

p_block_order <- ggplot(
  block_plot_summary,
  aes(
    x = block,
    y = mean,
    group = block_order
  )
) +
  geom_line(
    linewidth = 0.8,
    colour = "black"
  ) +
  geom_errorbar(
    aes(
      ymin = mean - se,
      ymax = mean + se
    ),
    width = 0.08,
    colour = "black",
    linewidth = 0.5
  ) +
  geom_point(
    aes(
      colour = time_pressure,
      shape = time_pressure
    ),
    size = 3.2
  ) +
  scale_colour_manual(
    values = c(
      "Self-paced" = "#E78AC3",
      "Time-limited" = "#4C78A8"
    )
  ) +
  scale_shape_manual(
    values = c(
```

```
      "Self-paced" = 17,
      "Time-limited" = 16
    )
  ) +
  facet_grid(
    measure ~ format,
    scales = "free_y",
    switch = "y"
  ) +
  labs(
    x = NULL,
    y = NULL,
    colour = "Condition",
    shape = "Condition"
  ) +
  theme_classic(base_size = 12) +
  theme(
    legend.position = "top",
    legend.title = element_blank(),

    strip.background = element_blank(),
    strip.placement = "outside",
    strip.text = element_text(
      face = "bold",
      size = 11
    ),
    strip.text.y.left = element_text(angle = 0),

    axis.text.x = element_text(size = 10),

    panel.spacing.y = grid::unit(1.5, "lines"),
    panel.spacing.x = grid::unit(0.8, "lines"),

    panel.border = element_rect(
      colour = "grey80",
      fill = NA,
      linewidth = 0.4
    ),

    plot.margin = margin(10, 15, 10, 15)
  )

dir.create(
  here::here("fig"),
  showWarnings = FALSE,
  recursive = TRUE
)

ggsave(
  filename = here::here(
    "fig",
    "Exp2_Block_Order_r05.png"
  ),
```

```
  plot = p_block_order,
  width = 12,
  height = 8,
  units = "in",
  dpi = 600,
  bg = "white"
)

ggsave(
  filename = here::here(
    "fig",
    "Exp2_Block_Order_r05.tiff"
  ),
  plot = p_block_order,
  width = 12,
  height = 8,
  units = "in",
  dpi = 600,
  compression = "lzw",
  bg = "white"
)
```